Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)

КАФЕДРА «Информационная безопасность» (ИУ8)

КУРСОВАЯ РАБОТА

НА ТЕМУ:

Разработка защищенной БД информационной системы «Хлебозавод»

Студент	ИУ8-84		Е. Д. Девяткин
	(группа)	(подпись, дата)	(И.О. Фамилия)
Руководитель курсового проекта			Е. В. Глинская
		(подпись, дата)	(И.О. Фамилия)

Москва, 2026

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

УТВЕРЖДАЮ

Заведующий кафедрой ИУ8
(индекс)
М.А. Басараб
(И.О. Фамилия)
_____ (подпись) 24.02.2026
(дата)

З А Д А Н И Е
на выполнение курсовой работы

по дисциплине Основы построения защищенных баз данных
Студент группы ИУ8-84 Девяткин Егор Дмитриевич
(Фамилия, имя, отчество)
Тема курсовой работы Разработка защищенной БД информационной системы «Хлебозавод».

Направленность КР (учебная, исследовательская, практическая, производственная, др.) учебно-практическая

Источник тематики (кафедра, предприятие, НИР) кафедра ИУ8

Задание Проанализировать существующую проблему информационной безопасности и защиты информации в базах данных информационной системы «Хлебозавод».
Представить решение проблемы информационной безопасности и защиты информации в базах данных информационной системы «Хлебозавод».

Оформление курсовой работы:
Расчетно-пояснительная записка (Отчет по КР) на _ _ листах формата А4.
Перечень графического (иллюстративного) материала (чертежи, плакаты, слайды и т.п.)
Презентация
Флешка
Дата выдачи задания « 24_ » февраля 2026 г.

Руководитель курсовой работы _____ Е.В. Глинская
(подпись, дата) (И.О. Фамилия)

Студент _____ Е.Д. Девяткин
(подпись, дата) (И.О. Фамилия)

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

КАЛЕНДАРНЫЙ ПЛАН
на выполнение курсовой работы

по дисциплине Основы построения защищенных баз данных
Студент группы ИУ8-84 Девяткин Егор Дмитриевич
(Фамилия, имя, отчество)
Тема курсовой работы Разработка защищенной БД информационной системы «Хлебозавод».

№ п/п	Наименование этапов выпускной квалификационной работы	Сроки выполнения этапов		Отметка о выполнении	
		план	факт	Руководитель КР	Куратор
1.	Задание на выполнение курсовой работы	*24.02.2026* Планируемая дата			
2.	Анализ предметной области; Проектирование системы базы данных	*05.04.2026* Планируемая дата			
3.	Реализация системы базы данных; Разработка и реализация средств защиты БД	*26.04.2026* Планируемая дата			
4.	Оформление РПЗ (Отчета)	*10.05.2026* Планируемая дата			
5.	Подготовка доклада и презентации	*10.05.2026* Планируемая дата			
6.	Защита курсовой работы	*16.05.2026* Планируемая дата			

Студент _____ (подпись, дата)
Руководитель работы _____ (подпись, дата)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ 5
ОСНОВНАЯ ЧАСТЬ 6
1 Анализ предметной области 6
2 Построение инфологической модели 10
3 Проектирование логической структуры системы 12
4 Проектирование базы данных 14
5 Реализация базы данных 18
6 Реализация уровня приложения 20
7 Реализация интерфейса 21
8 Реализация средств защиты БД 25
9 Отчёты 35
ЗАКЛЮЧЕНИЕ 36
СПИСОК ЛИТЕРАТУРЫ 37

ВВЕДЕНИЕ

В современных автоматизированных системах данные являются ключевым производственным ресурсом. Для хлебозавода защита базы данных особенно важна, поскольку в системе обрабатываются сведения о клиентах, поставщиках, сырье, продукции, заказах, договорах и счетах. Нарушение целостности таких данных может привести к некорректному расчету потребности в сырье, ошибочным поставкам, финансовым потерям и несанкционированному доступу к коммерческой информации.

Актуальность разработки защищенной базы данных обусловлена необходимостью контролировать доступ к документам и справочникам, предотвращать SQL-инъекции, хранить пароли в защищенном виде и ограничивать действия пользователей в зависимости от их роли. Общие требования к проектированию защищенных баз данных и разграничению доступа рассмотрены в учебных и методических источниках [1], [2], [3].

Целью курсовой работы является проектирование и реализация защищенной базы данных информационной системы хлебозавода, обеспечивающей хранение производственных и финансовых данных, ролевое разграничение доступа и контроль целостности.

Для достижения цели решаются задачи анализа предметной области, построения инфологической и физической моделей БД, реализации PostgreSQL-схемы, разработки прикладного уровня на FastAPI, настройки ограничений целостности, защиты от SQL-инъекций, хэширования паролей, ролевой модели доступа, аудита и резервного копирования [4], [5], [6], [7].

Объектом автоматизации является деятельность хлебозавода, связанная с учетом поставщиков, сырья, рецептур, продукции, клиентов, заказов, договоров и счетов. Предметом исследования выступают методы проектирования и защиты реляционной базы данных при наличии нескольких категорий пользователей.

ОСНОВНАЯ ЧАСТЬ

1 Анализ предметной области

1.1 Описание предметной области

Хлебозавод представляет собой предприятие пищевой промышленности, где производственный процесс связан с закупкой сырья, хранением его остатков, использованием рецептур, выпуском продукции, оформлением заказов клиентов, заключением договоров и выставлением счетов. Информационная система должна обеспечивать согласованное хранение данных об этих операциях и исключать противоречия между справочниками и документами.

В системе автоматизируются следующие процессы:

– ведение поставщиков и контактных данных ответственных лиц;
– ведение справочника сырья с количеством, единицами измерения, сроком годности и минимальным остатком;
– описание рецептов, связывающих продукцию с используемым сырьем, временем выпечки и температурой;
– учет готовой продукции, ее количества, срока годности и цены;
– ведение клиентов, контактных данных и адресов доставки;
– оформление заказов клиентов на продукцию;
– заключение договоров и фиксация статуса исполнения;
– выставление счетов по договорам и контроль сумм.

Рисунок 1 — Бизнес-процесс хлебозавода

1.2 Роли пользователей и разграничение доступа

В приложении используются логические роли admin, client, technologist, warehouse_worker и accountant. Роль не только определяет видимость разделов интерфейса, но и проверяется на сервере перед выполнением операций изменения данных. Такой подход соответствует принципу ограничения доступа к данным и операциям по полномочиям пользователя [2], [5].

Роль	**Назначение**	**Разрешенные действия**
admin	Администрирование системы	управление справочниками, пользователями, договорами, счетами и отчетами
client	Работа с собственными заказами	просмотр продукции, создание заказа, просмотр своих договоров и счетов
technologist	Ведение рецептур и продукции	создание и изменение рецептов, контроль состава продукции
warehouse_worker	Работа с поставщиками и сырьем	ведение сырья, поставщиков и остатков
accountant	Финансовый контур	работа со счетами, суммами и платежными статусами

Таблица 1 — Назначение логических ролей

Раздел	Полный доступ	Ограниченный доступ	Запрет
Поставщики	admin, warehouse_worker	technologist: просмотр	client, accountant
Сырье	admin, warehouse_worker	technologist: просмотр	client, accountant
Рецепты	admin, technologist	warehouse_worker: просмотр сырья	client, accountant
Продукция	admin, technologist	client: просмотр каталога	—
Клиенты	admin	client: только собственные данные	warehouse_worker, technologist
Заказы	admin	client: только собственные заказы	warehouse_worker
Договоры	admin, accountant	client: только свои договоры	warehouse_worker, technologist
Счета	admin, accountant	client: только свои счета	warehouse_worker, technologist

Таблица 2 — Матрица доступа к основным разделам

1.3 Информационные ресурсы и угрозы

К защищаемым информационным ресурсам относятся данные поставщиков, клиентов, сведения о сырье, рецептах, продукции, заказах, договорах, счетах и учетных записях пользователей. Для этих данных требуется обеспечить конфиденциальность, целостность и доступность. Наиболее опасными угрозами являются несанкционированный вход, изменение чужих заказов, подмена суммы счета, изменение цены продукции, SQL-инъекции, удаление связанных записей и потеря данных при сбое.

Ресурс	Возможная угроза	Средство защиты
Учетные данные	раскрытие пароля или подбор учетной записи	bcrypt-хэширование, проверка входа
Клиенты и поставщики	просмотр или изменение без полномочий	роли и серверные проверки действий

Сырье и продукция	отрицательные остатки, некорректные сроки годности	CHECK-ограничения и серверная валидация
Заказы и договоры	подмена статуса или чужого идентификатора	object-level access control
Счета	подмена суммы или повторное выставление счета	права роли accountant/admin, ограничения БД
SQL-запросы	SQL-инъекция	параметризованные запросы

Таблица 3 — Защищаемые ресурсы и основные угрозы

1.4 Входные и выходные документы

Документ	**Тип**	**Назначение**
Заказ клиента	входной/операционный	фиксация потребности клиента в продукции
Договор	операционный/правовой	закрепление условий взаимодействия с клиентом или поставщиком
Счет	выходной/финансовый	фиксация суммы к оплате по договору
Рецепт	нормативный	описание состава продукции и параметров выпечки
Справочник сырья	операционный	учет сырья, количества, единиц измерения и срока годности
Справочник продукции	операционный	учет готовой продукции, цены и срока годности

Таблица 4 — Входные и выходные документы системы

2 Построение инфологической модели

Инфологическая модель отражает состав сущностей предметной области и связи между ними без привязки к конкретной СУБД. В системе выделены восемь основных сущностей: поставщик, сырье, рецепт, продукция, клиент, заказ, договор и счет. Эта структура соответствует физической модели, представленной далее в разделе 4.

Сущность	**Назначение**	**Ключевые связи**
Поставщик	хранение информации о компаниях, поставляющих сырье	1:M с сырьем, связь с договорами
Сырье	учет материалов, используемых в производстве	M:1 с поставщиком, M:N с продукцией через рецепты
Рецепт	описание состава продукции и параметров выпечки	M:1 с продукцией, M:1 с сырьем
Продукция	учет готовых изделий хлебозавода	1:M с рецептами, 1:M с заказами
Клиент	хранение контактных данных покупателей	1:M с заказами, договорами и счетами
Заказ	фиксация заказанной продукции и количества	M:1 с клиентом и продукцией
Договор	учет договорных отношений	связан с участником сделки и счетами
Счет	учет финансовых начислений по договору	M:1 с договором и клиентом

Таблица 5 — Основные сущности инфологической модели

2.1 Связи между сущностями

Поставщик поставляет сырье; одна запись сырья относится к одному поставщику. Рецепт включает конкретное сырье и относится к конкретному виду продукции, поэтому через таблицу рецептов реализуется связь между продукцией и сырьем. Клиент получает заказы: каждый заказ связан с клиентом и выбранной продукцией. Клиент также заключает договоры и получает счета.

Счет связан с договором и клиентом, что позволяет контролировать финансовые документы по конкретному контрагенту.

Такая структура отделяет справочники от операционных документов и позволяет применять внешние ключи для поддержки ссылочной целостности [3].

3 Проектирование логической структуры системы

В системе реализуется как трехуровневая информационная система. Уровень представления содержит веб-страницы для входа, просмотра справочников, заказов, договоров, счетов и отчетов. Уровень приложения реализован на FastAPI и выполняет авторизацию, проверку ролей, обработку форм, параметризованные SQL-запросы и бизнес-валидацию. Уровень хранения данных представлен PostgreSQL, где размещаются таблицы физической модели, ключи, ограничения, индексы и служебные объекты защиты [6], [7].

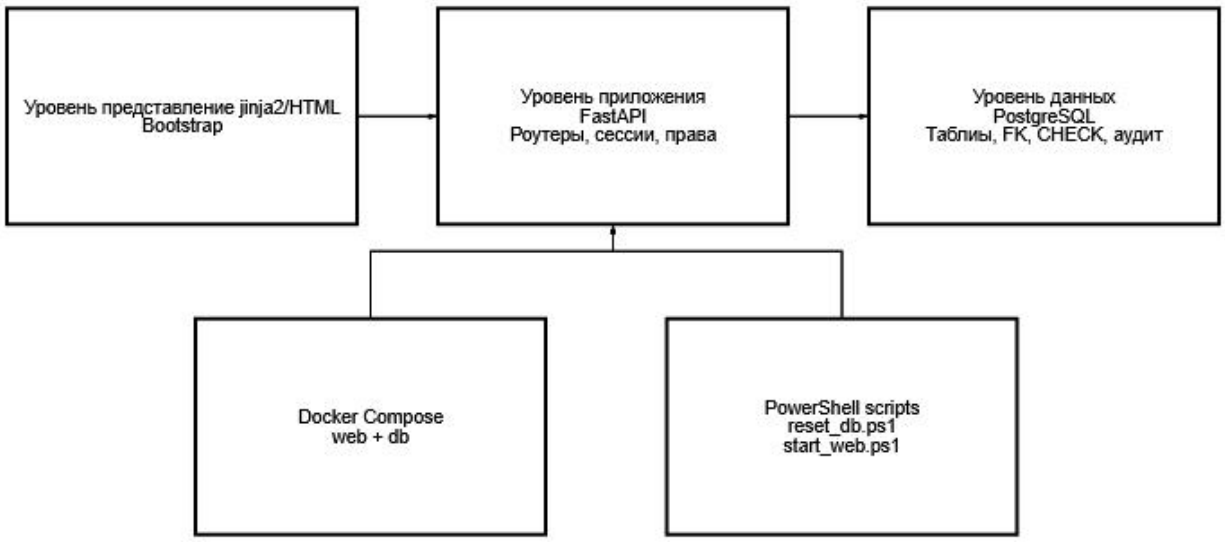

Рисунок 2 — Архитектура приложения

Модуль	**Назначение**	**Основные данные**
auth	вход, выход, проверка пароля и роли	учетные записи пользователей
suppliers/materials	ведение поставщиков и сырья	Поставщик, Сырье
recipes/products	ведение продукции и рецептов	Продукция, Рецепты
orders	создание и просмотр заказов	Заказы, Клиенты, Продукция
contracts/invoices	договоры и счета	Договора, Счета
reports	формирование сводных отчетов	данные заказов, сырья, продукции и счетов

Таблица 6 — Основные модули прикладного уровня

4 Проектирование базы данных

Проектирование БД выполнено с переходом от инфологической модели к физической модели PostgreSQL. Физическая модель задает конкретные таблицы, поля, первичные ключи, внешние ключи и основные связи между объектами. В представленной физической модели хлебозавода содержится 8 таблиц: Поставщик, Сырье, Рецепты, Продукция, Заказы, Клиенты, Договора и Счета.

4.1 Физическая модель базы данных

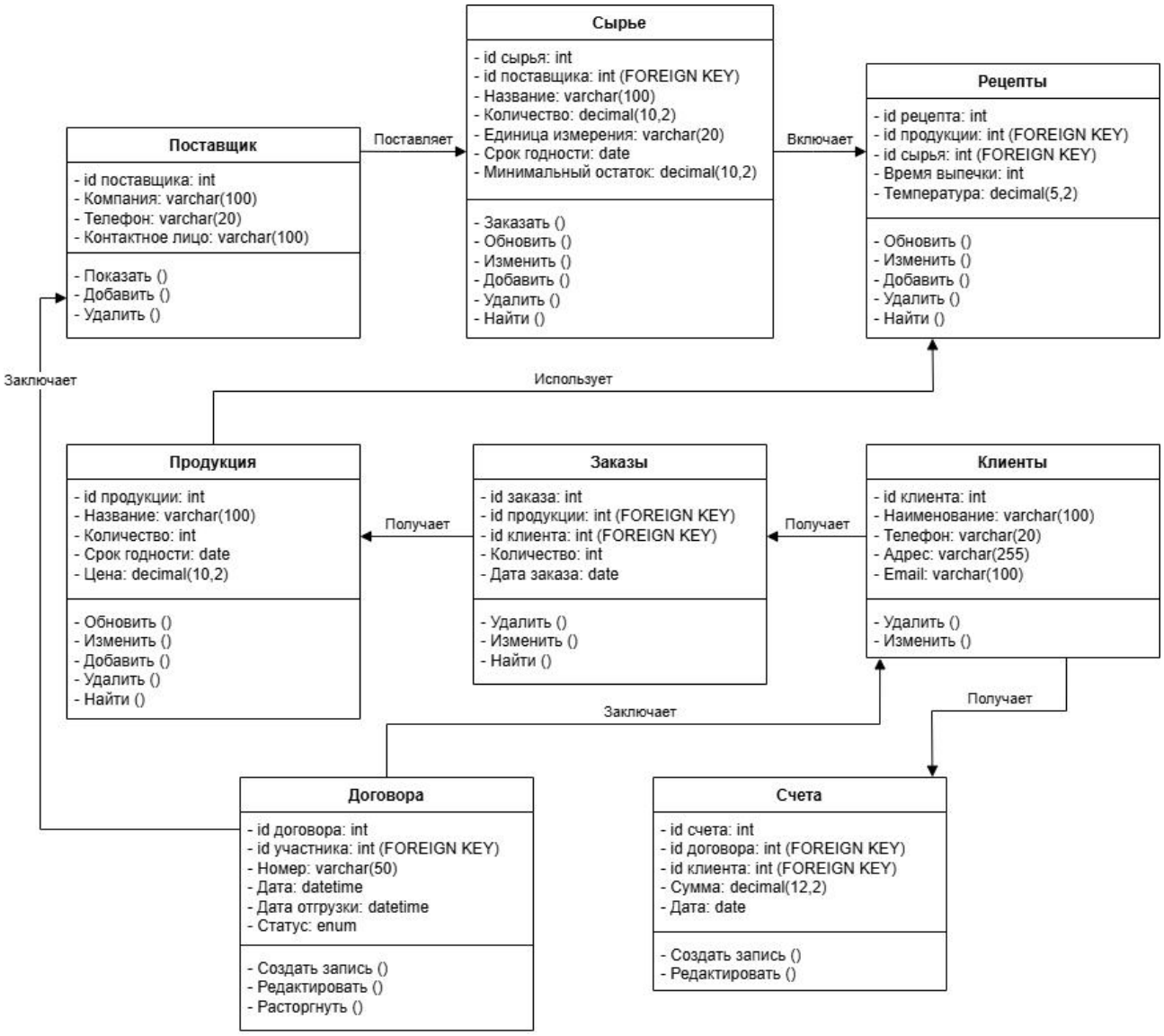

Рисунок 3 — Физическая модель базы данных хлебозавода

Физическая модель на рисунке 3 описывает учет движения сырья, рецептур, продукции, заказов и финансовых документов. Центральными

производственными таблицами являются «Сырье», «Рецепты» и «Продукция». Документарный контур представлен таблицами «Заказы», «Договора» и «Счета». Контрагенты разделены на поставщиков и клиентов.

Таблица	**Основные поля**	**Назначение и связи**
Поставщик	id_поставщика, компания, телефон, контактное_лицо	хранит поставщиков; связан с сырьем и договорами
Сырье	id_сырья, id_поставщика, название, количество, ед_измерения, срок_годности, минимальный_остаток	хранит сырье; id_поставщика является внешним ключом на поставщика
Рецепты	id_рецепта, id_продукции, id_сырья, время_выпечки, температура	связывает продукцию и сырье; содержит параметры выпечки
Продукция	id_продукции, название, количество, срок_годности, цена	хранит готовые изделия хлебозавода
Заказы	id_заказа, id_продукции, id_клиента, количество, дата_заказа	фиксирует заказ клиента на конкретную продукцию
Клиенты	id_клиента, наименование, телефон, адрес, email	хранит покупателей и контактные данные
Договора	id_договора, id_участника, номер, дата, дата_отгрузки, статус	фиксирует договорные отношения и статус исполнения
Счета	id_счета, id_договора, id_клиента, сумма, дата	хранит финансовые документы, связанные с договором и клиентом

Таблица 7 — Описание таблиц физической модели

Первичные ключи имеют тип integer. Внешние ключи используются для связей «Сырье — Поставщик», «Рецепты — Продукция», «Рецепты — Сырье», «Заказы — Продукция», «Заказы — Клиенты», «Счета — Договора» и «Счета — Клиенты». Поле id_участника в таблице «Договора» используется для связи

договора с участником сделки, что соответствует связям от поставщика и клиента на физической модели.

4.2 Ограничения целостности

Ограничения целостности обеспечивают корректность данных на уровне СУБД. Для физической модели хлебозавода применяются первичные ключи, внешние ключи, UNIQUE-ограничения, CHECK-ограничения и обязательные поля [3], [6].

Объект	**Ограничение**	**Назначение**
Поставщик	PRIMARY KEY(id_поставщика)	уникальная идентификация поставщика
Клиенты	UNIQUE(email)	запрет повторной регистрации клиента с тем же email
Сырье	FOREIGN KEY(id_поставщика)	сырье не может ссылаться на несуществующего поставщика
Сырье	CHECK(количество >= 0), CHECK(минимальный_остаток >= 0)	запрет отрицательных остатков
Рецепты	FOREIGN KEY(id_продукции), FOREIGN KEY(id_сырья)	рецепт может использовать только существующую продукцию и сырье
Продукция	CHECK(количество >= 0), CHECK(цена >= 0)	запрет отрицательного количества и цены
Заказы	FOREIGN KEY(id_продукции), FOREIGN KEY(id_клиента)	заказ связан с существующим клиентом и продукцией
Договора	CHECK(статус IN ('draft','signed','closed','cancelled'))	контроль допустимых статусов договора
Счета	FOREIGN KEY(id_договора), FOREIGN KEY(id_клиента), CHECK(сумма >= 0)	счет связан с договором и не может иметь отрицательную сумму

Таблица 8 — Основные ограничения целостности

```
ALTER TABLE raw_materials
ADD CONSTRAINT fk_raw_materials_supplier
FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id);

ALTER TABLE products
ADD CONSTRAINT chk_products_price
CHECK (price >= 0);

ALTER TABLE orders
ADD CONSTRAINT fk_orders_customer
FOREIGN KEY (customer_id) REFERENCES customers(customer_id);
```

4.3 Бизнес-правила модели

Для предметной области хлебозавода выделены бизнес-правила, которые должны соблюдаться при работе с таблицами физической модели. Поставщик не может быть удален при наличии связанного сырья; сырье с отрицательным количеством не допускается; рецепт не может ссылаться на несуществующую продукцию или сырье; заказ не может быть создан без клиента и продукции; счет не может быть выставлен без договора; сумма счета должна быть неотрицательной; статус договора должен принимать только предусмотренные значения.

Правило	**Место реализации**	**Результат нарушения**
запрет отрицательного количества сырья	CHECK в таблице raw_materials	операция INSERT/UPDATE отклоняется
запрет заказа на несуществующую продукцию	FK orders.product_id	заказ не создается
запрет счета без договора	FK invoices.contract_id	счет не создается
запрет недопустимого статуса договора	CHECK contracts.status	изменение статуса отклоняется
запрет повторяющегося email клиента	UNIQUE customers.email	запись не добавляется

Таблица 9 — Бизнес-правила физической модели

5 Реализация базы данных

База данных реализована в PostgreSQL. Прикладной уровень использует FastAPI, а веб-интерфейс формируется через Jinja2-шаблоны. Эти инструменты не выделяются в отдельную главу, поскольку в работе они рассматриваются в составе конкретных этапов реализации: PostgreSQL применяется при создании таблиц и ограничений, FastAPI — при реализации серверной логики и защиты маршрутов, Jinja2 — при построении экранных форм [6], [7].

5.1 SQL-скрипты и состав БД

Реализация БД организована набором SQL-скриптов: 00_drop.sql удаляет объекты для повторного запуска; 01_schema.sql создает таблицы физической модели; 02_constraints.sql подключает ключи и проверки; 03_indexes.sql создает индексы; 04_security_roles.sql настраивает роли и права; 05_audit.sql описывает журналирование; 06_seed.sql наполняет систему тестовыми данными; 07_test_queries.sql содержит проверочные запросы; 99_run_all.sql объединяет запуск.

Скрипт	**Назначение**
00_drop.sql	очистка схемы перед повторным запуском
01_schema.sql	создание таблиц поставщиков, сырья, рецептов, продукции, клиентов, заказов, договоров и счетов
02_constraints.sql	создание внешних ключей, CHECK и UNIQUE-ограничений
03_indexes.sql	создание индексов для ускорения поиска и контроля уникальности
04_security_roles.sql	настройка ролей СУБД и минимально необходимых прав
05_audit.sql	создание служебных объектов журналирования
06_seed.sql	загрузка тестовых данных
07_test_queries.sql	проверочные SELECT/INSERT/UPDATE-запросы

99_run_all.sql	последовательный запуск основных скриптов

Таблица 10 — SQL-скрипты базы данных

5.2 Примеры SQL-запросов

Тип	**Назначение**	**SQL-код**
Ввод	добавление клиента	INSERT INTO customers (name, phone, address, email) VALUES ('ООО Ромашка', '+7-900-000-00-01', 'Москва', 'client@example.ru');
Ввод	добавление поставщика	INSERT INTO suppliers (company, phone, contact_person) VALUES ('Мельница №1', '+7-900-000-00-02', 'Петров П.П.');
Выборка	заказы клиента	SELECT o.order_id, p.name, o.quantity FROM orders o JOIN products p ON p.product_id=o.product_id WHERE o.customer_id=:customer_id;
Группировка	количество заказов по продукции	SELECT p.name, COUNT(*) FROM orders o JOIN products p ON p.product_id=o.product_id GROUP BY p.name;
Агрегат	сумма счетов клиента	SELECT SUM(amount) FROM invoices WHERE customer_id=:customer_id;

Таблица 11 — Примеры SQL-запросов к базе хлебозавода

6 Реализация уровня приложения

Уровень приложения реализован на FastAPI. Серверные маршруты принимают данные из форм, проверяют авторизацию, выполняют проверку роли пользователя, формируют параметризованные SQL-запросы и возвращают HTML-страницы. На этом уровне реализуются дополнительные правила, которые нельзя надежно обеспечить только скрытием кнопок в интерфейсе: запрет доступа клиента к чужим счетам, запрет изменения цены клиентом, запрет выполнения административных действий без роли admin.

Модуль	**Назначение**
main.py	создание приложения FastAPI, подключение middleware, шаблонов и маршрутов
auth.py	авторизация, выход, проверка bcrypt-хэша пароля
permissions.py	описание SECTION_RULES, ACTION_RULES и функции authorize_action
db.py	подключение к PostgreSQL и выполнение параметризованных запросов
suppliers.py/materials.py	работа с поставщиками и сырьем
recipes.py/products.py	работа с рецептами и продукцией
orders.py	создание и просмотр заказов
contracts.py/invoices.py	работа с договорами и счетами
reports.py	формирование отчетов

Таблица 12 — Основные модули приложения

7 Реализация интерфейса

Интерфейс реализован как серверно-рендерируемое веб-приложение на Jinja2-шаблонах. Навигационное меню строится в соответствии с ролью текущего пользователя. При этом скрытие разделов в интерфейсе не считается самостоятельной защитой: каждое критичное действие дополнительно проверяется на сервере.

Форма	**Назначение**	**Доступ**
login	вход пользователя в систему	все пользователи
dashboard	главная панель пользователя	все авторизованные пользователи
suppliers	поставщики	admin, warehouse_worker
materials	сырье	admin, warehouse_worker, technologist: просмотр
recipes	рецепты	admin, technologist
products	продукция	все авторизованные пользователи с учетом роли
orders	заказы	admin, client
contracts	договоры	admin, accountant, client: свои
invoices	счета	admin, accountant, client: свои
reports	отчеты	admin, accountant

Таблица 13 — Основные экранные формы

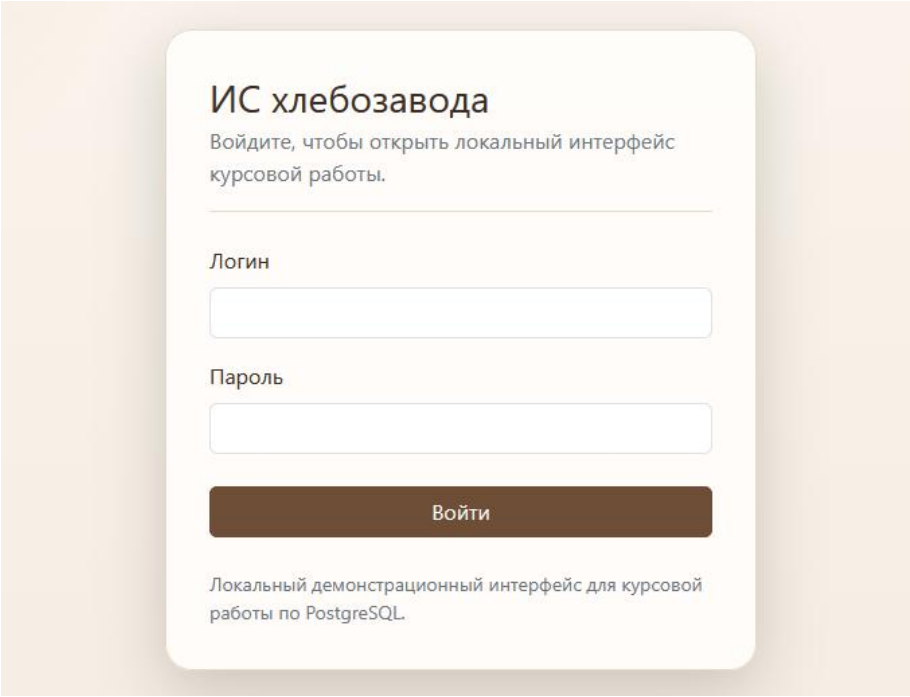

Рисунок 4 — Форма входа в систему

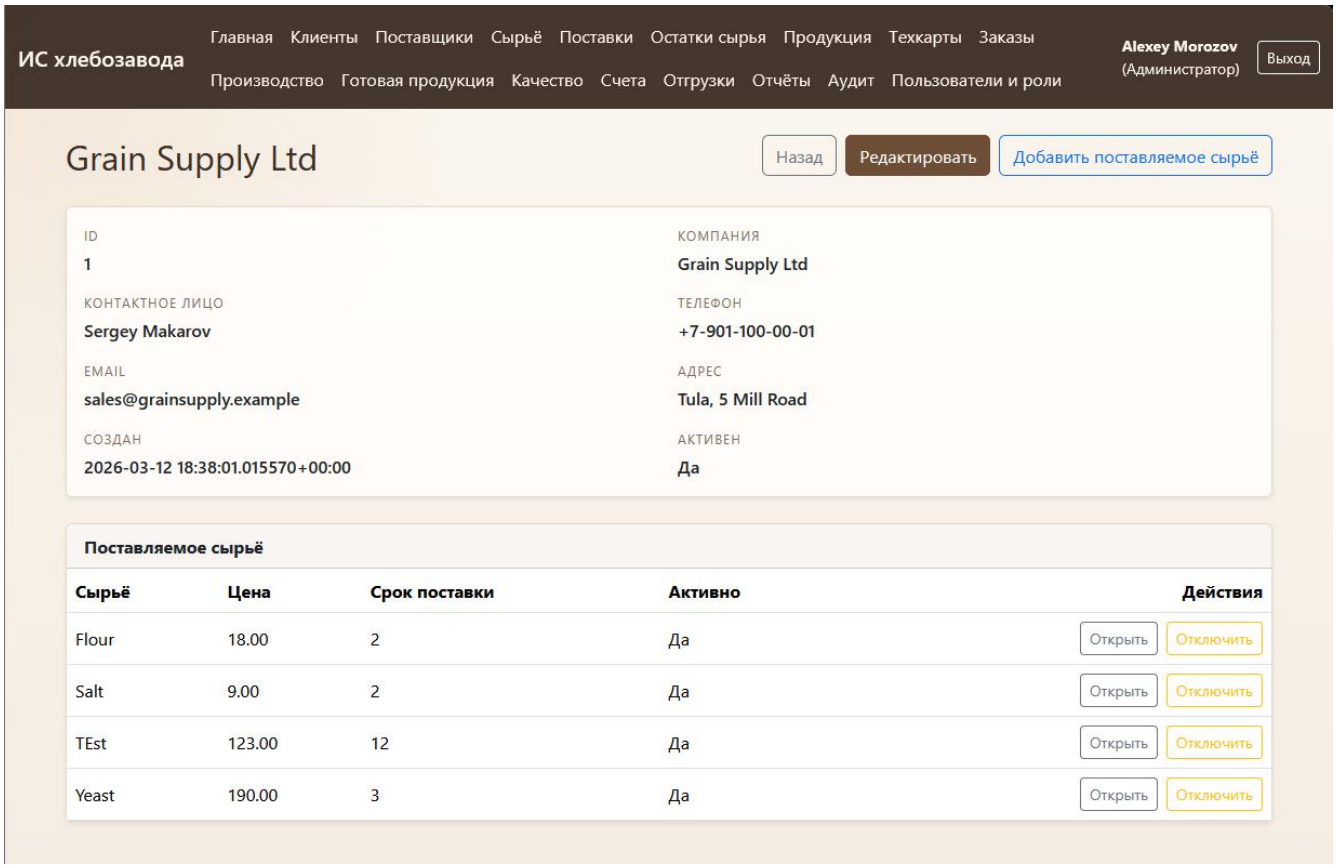

Рисунок 5 — Главное меню пользователя с ролью admin

ИС хлебозавода Главная Продукция Заказы Счета Отгрузки Ivan Petrov (Клиент) Выход

Продукция

Справочник готовой продукции.

ID	Продукция	Категория	Ед.	Цена	Срок годности	Активна	Доступно	Действия
1	Borodinsky Bread	Bread	piece	85.00	5	Да	16.000	Открыть
2	Moscow Bread	Bread	piece	78.00	4	Да	0	Открыть
3	Poppy Seed Roll	Pastry	piece	140.00	4	Да	30.000	Открыть
4	Bulka	Bread	piece	123.00	12	Да	9.000	Открыть

Рисунок 6 — Главное меню пользователя с ролью client

ИС хлебозавода Главная Продукция Заказы Счета Отгрузки Ivan Petrov (Клиент) Выход

Создать заказ Назад

Плановая дата отгрузки

дд.мм.гггг

Комментарий

Создать заказ Отмена

ИС хлебозавода Главная Продукция Заказы Счета Отгрузки Ivan Petrov (Клиент) Выход

Добавить позицию в заказ #6 Назад

Продукция

Borodinsky Bread / доступно 16.000

Количество

Добавить позицию Отмена

Рисунок 7 — Форма работы с заказами

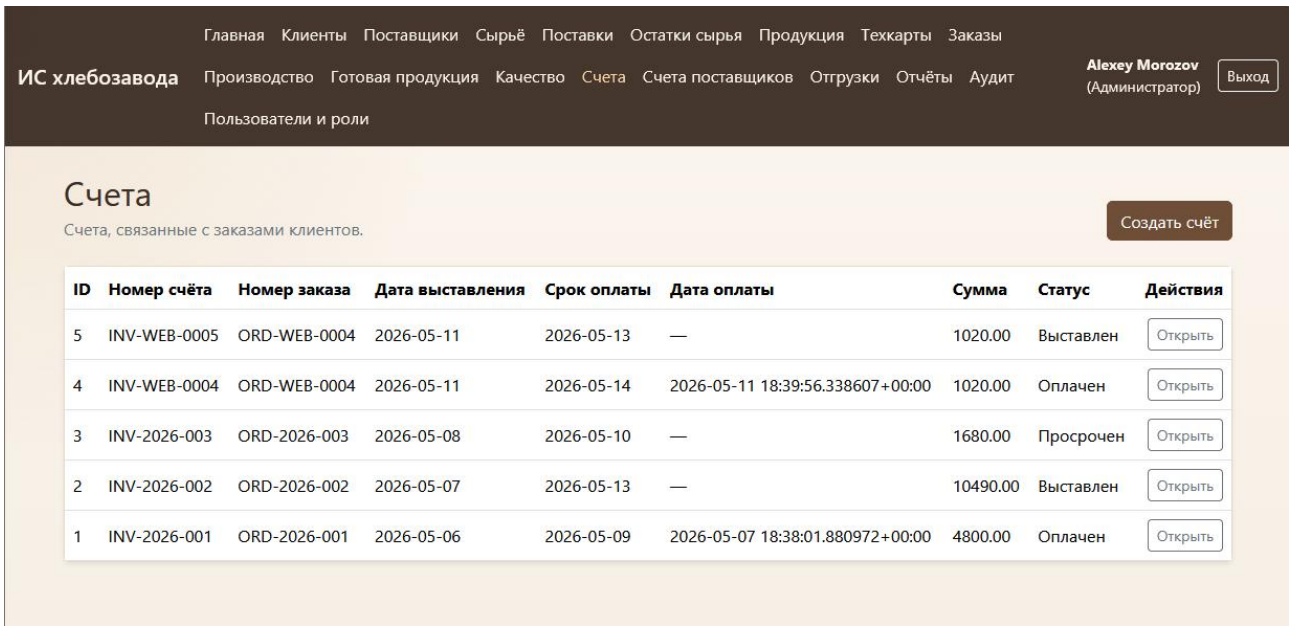

Рисунок 8 — Форма работы со счетами

8 Реализация средств защиты БД

Защита базы данных реализована как совокупность мер на уровне PostgreSQL и на уровне веб-приложения. На уровне БД применяются ключи, ограничения, индексы, роли, GRANT и резервное копирование. На уровне приложения реализованы авторизация, bcrypt-хэширование паролей, параметризованные запросы, проверки ролей и проверки принадлежности объектов текущему пользователю. Такой подход соответствует рекомендациям OWASP по предотвращению SQL-инъекций, безопасному хранению паролей и предотвращению нарушений контроля доступа [4], [5].

Угроза	**Реализованная защита**	**Пример результата**
несанкционированный вход	проверка логина и bcrypt-хэша пароля	неверные учетные данные не открывают систему
раскрытие пароля	пароль не хранится открытым текстом	в БД хранится bcrypt-хэш
SQL-инъекция	параметризованные SQL-запросы	ввод обрабатывается как данные, а не как SQL-код
доступ к чужим разделам	ACTION_RULES и authorize_action	client получает 403 при обращении к закрытым маршрутам
нарушение связей таблиц	FOREIGN KEY	невозможно создать заказ без клиента или продукции
некорректные числовые значения	CHECK-ограничения	отрицательная цена или количество отклоняются
потеря данных	резервное копирование pg_dump	создается резервная копия БД

Таблица 14 — Средства защиты БД

8.1 Авторизация пользователей

Авторизация реализована на уровне приложения. Пользователь вводит логин и пароль, после чего сервер получает запись учетной записи, проверяет хэш пароля и сохраняет в сессии идентификатор пользователя и его роль. При ошибке авторизации система не открывает защищенные разделы.

```
user = db.fetch_one(
    "SELECT user_id, username, password_hash, role_code FROM users WHERE username = :username",
    {"username": username}
)

if user is None or not pwd_context.verify(password, user["password_hash"]):
    raise HTTPException(status_code=401, detail="Неверный логин или пароль")
```

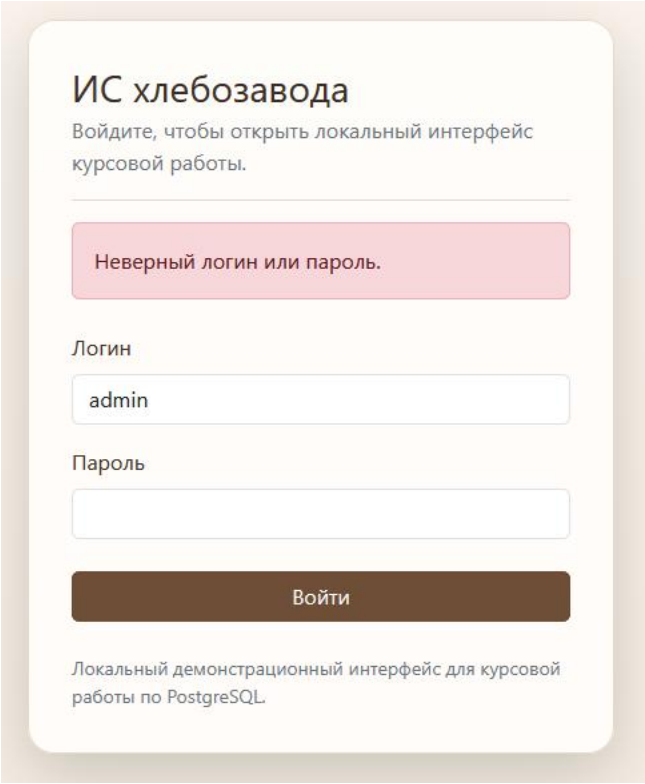

Рисунок 9 — Пример отказа во входе при неверных учетных данных

8.2 Разграничение доступа по ролям

Ролевая модель доступа реализована через набор разрешений для разделов и действий. Проверка выполняется на сервере перед выполнением чувствительных операций. Это исключает обход защиты через прямой URL или подмену POST-запроса, поскольку сервер повторно проверяет роль текущего пользователя [5].

```
ACTION_RULES = {
    "suppliers.edit": {"admin", "warehouse_worker"},
    "recipes.edit": {"admin", "technologist"},
    "orders.create": {"admin", "client"},
    "invoices.edit": {"admin", "accountant"},
    "reports.view": {"admin", "accountant"},
}

def authorize_action(user, action_code):
    if user.role_code not in ACTION_RULES.get(action_code, set()):
        raise  HTTPException(status_code=403,  detail="Недостаточно
прав")
```

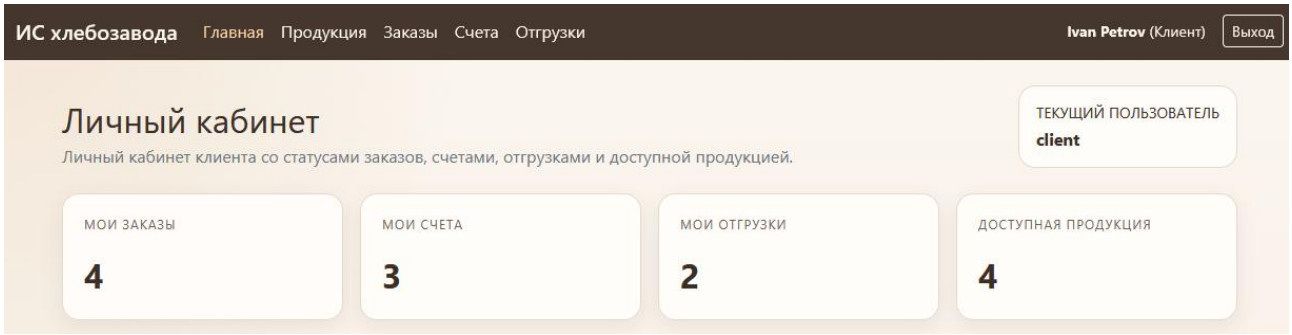

Рисунок 10 — Пример меню пользователя client без административных разделов

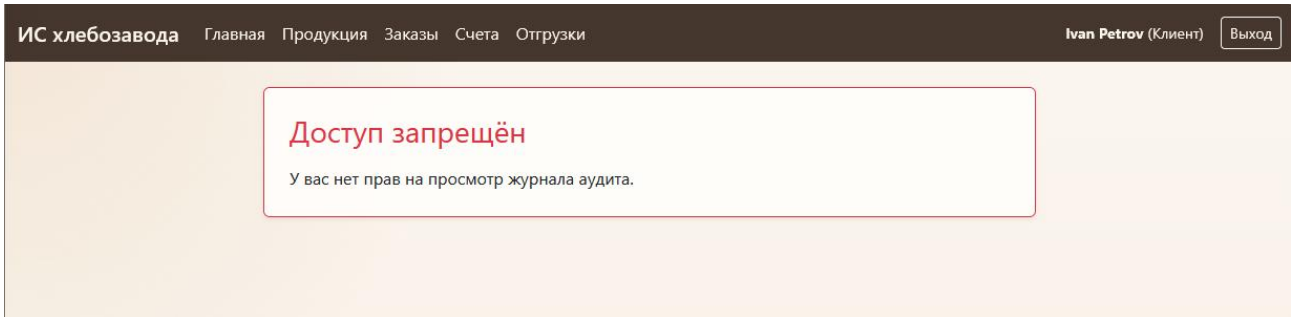

Рисунок 11 — Пример ошибки 403 при обращении client к закрытому разделу аудита

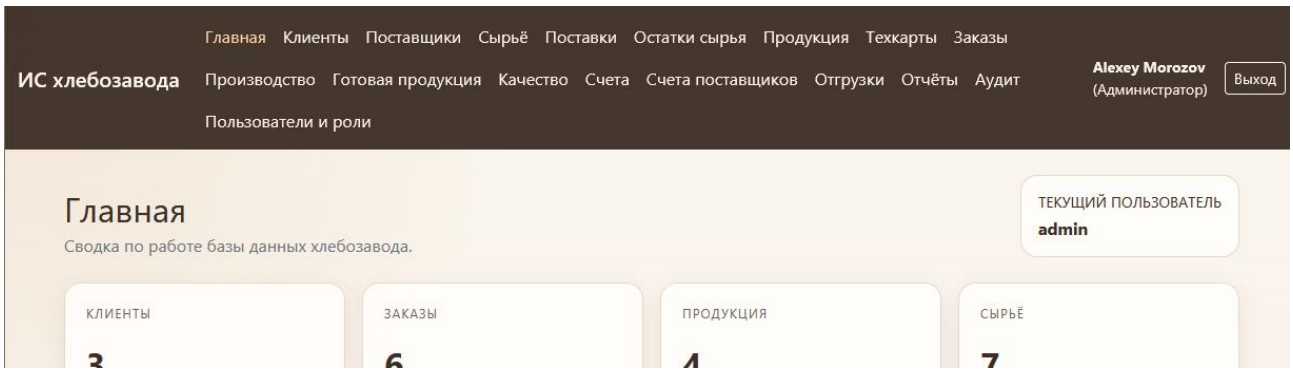

Рисунок 12 — Пример меню администратора

8.3 Хранение паролей

Пароли пользователей не сохраняются в базе данных в открытом виде. При регистрации или создании нового пользователя приложение получает введённый пароль только на уровне формы, после чего перед передачей в базу данных преобразует его в bcrypt-хэш. Для этого используется библиотека passlib, в которой создаётся объект CryptContext с алгоритмом bcrypt.

Алгоритм bcrypt формирует не просто хэш пароля, а защищённую строку, включающую служебную информацию об алгоритме, параметрах вычисления и случайной соли. Соль создаётся автоматически для каждого пароля, поэтому даже два одинаковых пароля у разных пользователей будут иметь разные значения хэша в таблице users. Это усложняет подбор паролей по заранее подготовленным таблицам хэшей.

После вычисления bcrypt-хэша исходный пароль в базу данных не записывается. В таблицу users сохраняется только значение password_hash. При последующей авторизации пользователь снова вводит пароль, приложение не расшифровывает старый хэш, а выполняет проверку: введённый пароль сравнивается с ранее сохранённым bcrypt-хэшем через функцию проверки. Если пароль соответствует хэшу, вход разрешается; если нет — пользователю возвращается отказ в авторизации.

```
from passlib.context import CryptContext
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
password_hash = pwd_context.hash(password)
# В таблицу users сохраняется password_hash, а не исходный пароль.
```

Также приведен пример проверки хэша при авторизации.

```
is_valid = pwd_context.verify(input_password, user.password_hash)
if not is_valid:
    raise HTTPException(status_code=401, detail="Неверный логин
или пароль")
```

```
   username   |                          password_hash
--------------+--------------------------------------------------------------
 admin        | $2a$12$jkKp8E0fanhlN/eJFXojXum4aixFdyzzr4yDF3srea0daReXAVGEO
 client       | $2a$12$z1IwsEkJBr5jNgi1oBCqMuoAkNt5IasBXe8HuBXVcOxrzEojWcXE.
 quality      | $2a$12$RhUwYfUPeOnWUEGDwLjfwObgqxXAQXHcEbZwM07TYH5HL.qX4taPO
 technologist | $2a$12$bFJBjKSrwto5057/5W0XhOoe5VI/ZIMKo1OY5VTvmtd1D2yx2Zbwq
 warehouse    | $2a$12$b8ckK1gpo6CH2.GdJljRx.npceD7hAu1jrhU79S.KCr3CTDhRx2du
(5 rows)
```

Рисунок 13 — Пример хранения bcrypt-хэша пароля в таблице пользователей

8.4 Защита от SQL-инъекций

Защита от SQL-инъекций реализована за счёт параметризованных SQL-запросов. Пользовательские значения не объединяются со строкой SQL через конкатенацию, а передаются отдельно от текста запроса через параметры. Поэтому строка вида ' OR 1=1 -- воспринимается системой как обычное значение поля, а не как часть SQL-команды [4].

Пример параметризованного запроса при авторизации пользователя:

```
query = """
    SELECT
        u.user_id,
        u.username,
        u.password_hash,
        u.full_name,
        u.email
    FROM users AS u
    WHERE u.username = %s
"""
row = fetch_one(query, (username,))
```

В данном фрагменте вместо прямой подстановки логина в SQL-запрос используется плейсхолдер %s. Значение переменной username передаётся отдельно в функцию fetch_one() как параметр (username,).

Общее выполнение запросов с параметрами реализовано в модуле работы с базой данных:

```
def fetch_one(query: str, params=None, user_id=None, user_ip=None):
    with get_db(user_id=user_id, user_ip=user_ip) as conn:
        with conn.cursor() as cur:
            cur.execute(query, params or ())
            return cur.fetchone()

def execute(query: str, params=None, user_id=None, user_ip=None) -
> int:
    with get_db(user_id=user_id, user_ip=user_ip) as conn:
        with conn.cursor() as cur:
            cur.execute(query, params or ())
            return cur.rowcount
```

Метод cur.execute(query, params or ()) получает отдельно текст SQL-запроса и отдельно значения параметров. За счёт этого пользовательский ввод не вставляется напрямую в SQL-строку, а обрабатывается как данные. Это не позволяет изменить структуру запроса через поля формы и предотвращает обход авторизации с помощью SQL-инъекций.

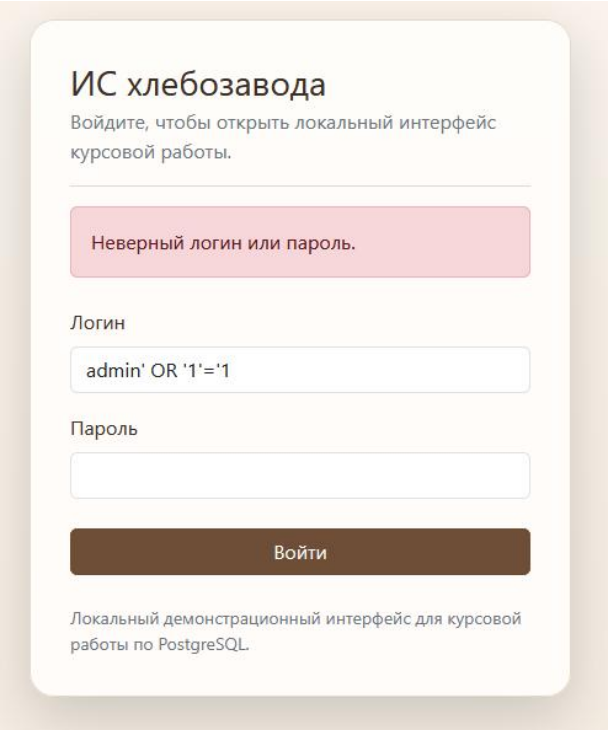

Рисунок 14 — Пример отказа при вводе SQL-инъекции в форму входа

8.5 Ограничения целостности

Ограничения целостности реализованы средствами PostgreSQL. Они предотвращают появление счетов с отрицательной суммой, договоров с недопустимым статусом, рецептов без связанного сырья или продукции, а также записей с нарушенными связями между таблицами. В отличие от проверки только на уровне интерфейса, ограничения БД срабатывают независимо от способа обращения к базе данных [3], [6].

```
ALTER TABLE invoices
ADD CONSTRAINT chk_invoices_amount CHECK (amount >= 0);

ALTER TABLE contracts
ADD CONSTRAINT chk_contracts_status
CHECK (status IN ('draft', 'signed', 'closed', 'cancelled'));

ALTER TABLE recipes
ADD CONSTRAINT fk_recipes_product
FOREIGN KEY (product_id) REFERENCES products(product_id);
```

```
bakery_security_db=# BEGIN;

UPDATE invoices
SET amount = -100
WHERE invoice_id = 1;
BEGIN
ERROR:  new row for relation "invoices" violates check constraint "chk_invoices_amount"
DETAIL:  Failing row contains (1, INV-2026-001, 1, 2026-05-06, 2026-05-09, 2026-05-07 18
 Paid immediately after shipment.).
bakery_security_db=!# ROLLBACK;
ROLLBACK
bakery_security_db=#
```

Рисунок 15 — Пример ошибки при нарушении CHECK-ограничения

На рисунке показана попытка установить отрицательную сумму счёта в таблице invoices. PostgreSQL отклоняет операцию, так как срабатывает CHECK-ограничение chk_invoices_amount, запрещающее значения amount < 0. Это подтверждает, что база данных контролирует корректность финансовых данных независимо от интерфейса приложения

```
bakery_security_db=# BEGIN;

UPDATE invoices
SET order_id = 999999999
WHERE invoice_id = 1;
BEGIN
ERROR:  insert or update on table "invoices" violates foreign key constraint "fk_invoices_order_id"
DETAIL:  Key (order_id)=(999999999) is not present in table "customer_orders".
bakery_security_db=!# ROLLBACK;
ROLLBACK
bakery_security_db=# |
```

Рисунок 16 — Пример ошибки при нарушении внешнего ключа

На рисунке показана попытка связать счёт с несуществующим заказом через значение order_id = 99999999. PostgreSQL отклоняет операцию из-за внешнего ключа fk_invoices_order_id, так как такого заказа нет в таблице customer_orders. Это подтверждает защиту ссылочной целостности между счетами и заказами.

8.6 Контроль операций и аудит

Для контроля действий пользователей используется журналирование операций. В журнал записываются пользователь, тип действия, таблица, время операции и измененные данные. Аудит позволяет установить, кто изменил справочник сырья, создал заказ, изменил договор или выставил счет. В

предметную физическую модель включены основные производственные и финансовые таблицы, а audit_log используется как служебный объект защиты.

```
INSERT INTO audit_log(user_id, action_type, table_name, new_data,
changed_at)
VALUES (:user_id, 'INSERT', 'orders', :new_data::jsonb, now());
```

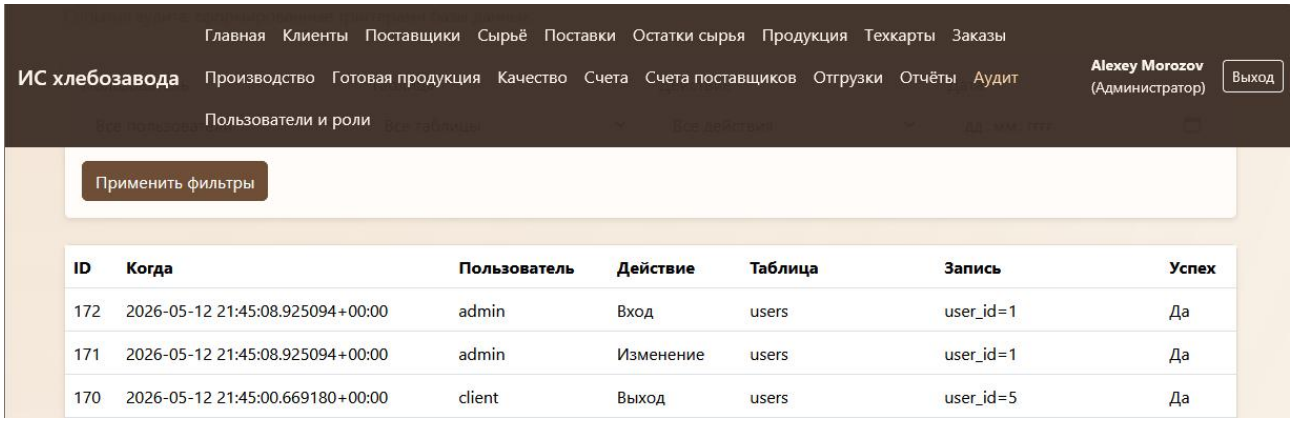

Рисунок 17 — Пример записи действия пользователя в журнале аудита

В целом также стоит заметить, что доступ к аудиту есть только у администратора.

8.7 Резервное копирование

Для защиты от потери данных используется резервное копирование PostgreSQL. Резервная копия создаётся командой pg_dump и содержит структуру базы данных, ограничения целостности, роли, таблицы и данные. Наличие резервной копии позволяет восстановить сведения о пользователях, клиентах, поставщиках, сырье, заказах, счетах, производстве, отгрузках и журнале аудита после сбоя [3].

```
pg_dump -h localhost -p 5432 -U postgres -d slop_bakery   -F c -f
backups/slop_bakery_full.backup
```

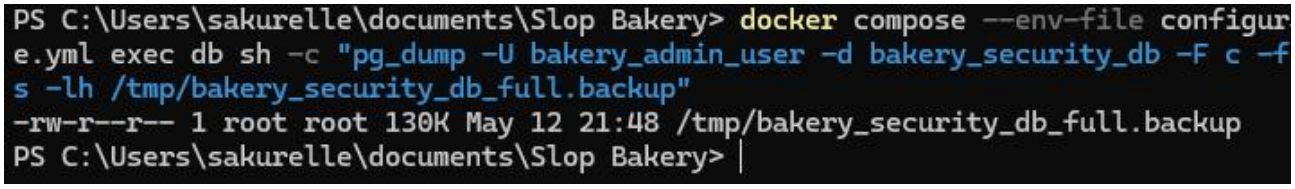

Рисунок 18 — Пример успешного создания резервной копии базы данных

На рисунке показано успешное создание резервной копии базы данных bakery_security_db. Команда pg_dump формирует backup-файл в формате custom (-F c), после чего команда ls -lh подтверждает наличие созданного файла и его размер. Это подтверждает, что резервное копирование БД выполнено корректно.

8.8 Демонстрационные примеры защиты

В системе реализованы демонстрационные примеры защиты, которые подтверждают работу механизмов безопасности. Для SQL-инъекции показан отказ во входе при вводе вредоносной строки в поле логина или пароля. Для хэширования показано, что в таблице пользователей хранится bcrypt-хэш, а не исходный пароль. Для ролевой модели показано различие интерфейса admin и client, а также возврат ошибки 403 при прямом обращении к закрытому маршруту. Для ограничений целостности показан отказ БД при попытке добавить отрицательное количество или запись с несуществующим внешним ключом.

Механизм	**Что реализовано**	**Подтверждение в отчете**
SQL-инъекции	параметризованные запросы во всех маршрутах с пользовательским вводом	рисунки 14
Хэширование паролей	bcrypt-хэш сохраняется вместо открытого пароля	рисунок 13
Ролевая модель доступа	server-side проверка ACTION_RULES	рисунки 11–12
Ограничения целостности	PK, FK, UNIQUE и CHECK на уровне PostgreSQL	рисунки 15–16
Аудит	фиксация значимых действий пользователей	рисунок 17
Резервное копирование	создание backup-файла PostgreSQL	рисунок 18

Таблица 15 — Фактические примеры реализации защиты

9 Отчёты

В системе предусмотрены отчеты, формируемые по данным физической модели. Доступ к отчетам ограничивается ролью пользователя: клиент видит только собственные документы, бухгалтер и администратор получают доступ к финансовым отчетам, технолог и складской работник используют производственные и складские данные.

Отчет	**Доступ**	**Содержание**
Остатки сырья	admin, warehouse_worker, technologist	название сырья, количество, единица измерения, минимальный остаток
Сырье ниже минимума	admin, warehouse_worker	позиции, где количество меньше минимального остатка
Каталог продукции	admin, client, technologist	название, количество, срок годности, цена
Заказы клиентов	admin, client: свои	клиент, продукция, количество, дата заказа
Договоры	admin, accountant, client: свои	номер, дата, дата отгрузки, статус
Счета	admin, accountant, client: свои	сумма, дата, договор, клиент

Таблица 16 — Реализованные отчеты

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы разработана защищенная база данных информационной системы хлебозавода. Предметная физическая модель описывает 8 таблиц: Поставщик, Сырье, Рецепты, Продукция, Заказы, Клиенты, Договора и Счета. Для этих таблиц определены ключевые поля, связи, первичные и внешние ключи, а также ограничения целостности.

Реализация выполнена с использованием PostgreSQL для хранения данных и FastAPI для прикладного уровня. В тексте реализации указано, какие инструменты применяются на каждом этапе: PostgreSQL используется для таблиц, ключей, ограничений и резервного копирования; FastAPI используется для маршрутов, авторизации, серверных проверок прав и параметризованных запросов; Jinja2 используется для веб-интерфейса.

В системе реализованы основные средства защиты БД: авторизация пользователей, хранение bcrypt-хэшей паролей, защита от SQL-инъекций с помощью параметризованных запросов, ролевая модель доступа, ограничения целостности, аудит действий пользователей и резервное копирование. Раздел с инструкциями удален; вместо него в разделе реализации защиты приведены факты реализации, фрагменты кода и места для скриншотов, подтверждающих работу механизмов защиты.

СПИСОК ЛИТЕРАТУРЫ

1. Боровик И. Г., Глинская Е. В. Проектирование защищенных систем баз данных: методические указания к курсовому проектированию. — М.: МГТУ им. Н. Э. Баумана, 2025.

2. Баранчиков А. И., Баранчиков П. А., Пылькин А. Н. Алгоритмы и модели ограничения доступа к записям баз данных. — М.: Горячая линия–Телеком, 2016. — 182 с.

3. Булдакова Т. И. Технологии разработки баз данных. Электронное учебное издание. — М.: МГТУ имени Н. Э. Баумана, 2012. — ФГУП «Информрегистр» № 0321203017.

4. OWASP Cheat Sheet Series. SQL Injection Prevention Cheat Sheet. — Электронный ресурс. — URL: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html .

5. OWASP Foundation. Broken Access Control. — Электронный ресурс. — URL: https://owasp.org/www-community/Broken_Access_Control .

6. PostgreSQL Global Development Group. PostgreSQL Documentation. Version 16. — Электронный ресурс. — URL: https://www.postgresql.org/docs/16/ .

7. FastAPI Documentation. — Электронный ресурс. — URL: https://fastapi.tiangolo.com/ .